



ΠΑΝΕΠΙΣΤΗΜΙΟ ΔΥΤΙΚΗΣ ΜΑΚΕΔΟΝΙΑΣ
Πολυτεχνική Σχολή
Τμήμα Ηλεκτρολόγων Μηχανικών & Μηχανικών Υπολογιστών

Ανάλυση και Σχεδίαση

Αλγορίθμων

Εργασία 3

Τσίγγερης Αντώνιος 2026

Ομάδα 8

06/06/2025

Καθηγητής: Νικόλαος Πλόσκας

```

kruskal.h  reverse_delete.h  main.c x
> Q-  Cc W .*  0 results  ↑ ↓ 🔍
42  int main() {
65      for(int k=0;k<4;k++)
95          for (int i = 0; i < numVertices; ++i) {
98              graphMatrix[i][temp->destination] = temp->baros;
99              temp = temp->nextlist;
100          }
101      }
102
103      printf( format:"==== Graph#%d =====\n",k+1); // Ektypwsi tou arithmou tou grafou
104      //printGraph(graph); // typonei olo to grafo
105
106      printf( format:"Prim : \n");
107      start1 = clock(); // Enarksi metrisis xronou gia Prim
108      primMST(graph, graphMatrix); // Ektelesi algorithmou Prim
109      end1 = clock(); // Lixi metrisis xronou gia Prim
110      cpu_time_used_for_prim[k]= (double)(end1 - start1) / (CLOCKS_PER_SEC); // Ypologismos xronou se deuterolepta
111      sum = sum + cpu_time_used_for_prim[k]; // Prosthiiki sto synoliko athroisma xronou gia Prim
112      printf( format:"-----\n");
113
114      printf( format:"Kruskal : \n");
115      start2 = clock(); // Enarksi metrisis xronou gia Kruskal
116      kruskaMinimum_spanning_tree(graphKruskal); // Ektelesi algorithmou Kruskal
117      end2 = clock(); // Lixi metrisis xronou gia Kruskal
118      cpu_time_used_for_kruskal[k]= (double)(end2 - start2) / (CLOCKS_PER_SEC); // Ypologismos xronou se deuterolepta
119      sum2 = sum2 + cpu_time_used_for_kruskal[k]; // Prosthiiki sto synoliko athroisma xronou gia Kruskal
120      printf( format:"-----\n");
121
122      printf( format:"Reverse-delete : \n");
123      start3 = clock(); // Enarksi metrisis xronou gia Reverse Delete
124      reverseDelete(graphReverse); // Ektelesi algorithmou Reverse Delete
125      end3 = clock(); // Lixi metrisis xronou gia Reverse Delete
126      cpu_time_used_for_Rreverse_delete[k]= (double)(end3 - start3) / (CLOCKS_PER_SEC); // Ypologismos xronou se deuterolepta
127      sum3 = sum3 + cpu_time_used_for_Rreverse_delete[k]; // Prosthiiki sto synoliko athroisma xronou gia Reverse Delete
128      printf( format:"-----\n");
129  }

```

Γράφοι και Αποτελέσματα :

Graph 1:

```

===== Graph#1 =====
Node 0: 0->4 W: 86
Node 0: 0->3 W: 1
Node 0: 0->2 W: 28
Node 0: 0->1 W: 46
Node 1: 1->3 W: 23
Node 1: 1->2 W: 35
Node 1: 1->0 W: 46
Node 2: 2->4 W: 36
Node 2: 2->3 W: 78
Node 2: 2->1 W: 35
Node 2: 2->0 W: 28
Node 3: 3->4 W: 93
Node 3: 3->2 W: 78
Node 3: 3->1 W: 23
Node 3: 3->0 W: 1
Node 4: 4->3 W: 93
Node 4: 4->2 W: 36
Node 4: 4->0 W: 86

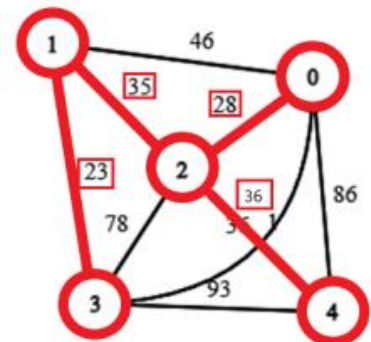
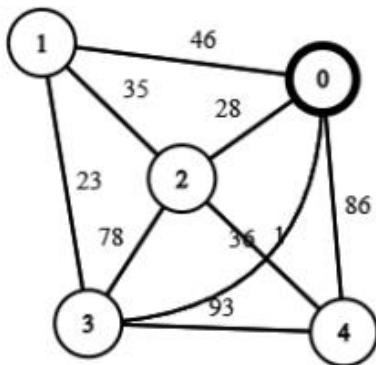
```

```

Prim :
The algorithm with the Adjacency Lists gives the following minimum spanning tree:
3 <--> 1 :weight: 23
0 <--> 2 :weight: 28
0 <--> 3 :weight: 1
2 <--> 4 :weight: 36
-----
Kruskal :

Edges minimum spanning tree created with Kruskal algorithm :
0 <--> 3      Weight: 1
3 <--> 1      Weight: 23
2 <--> 0      Weight: 28
2 <--> 4      Weight: 36
-----
Reverse-delete :
=====
Minimum Spanning Tree Edges:
2 <--> 4      Weight: 36
0 <--> 2      Weight: 28
1 <--> 3      Weight: 23
0 <--> 3      Weight: 1

```



Graph 2:

```

===== Graph#2 =====
Node 0: 0->4 W: 3
Node 0: 0->3 W: 3
Node 0: 0->2 W: 95
Node 0: 0->1 W: 24
Node 1: 1->2 W: 47
Node 1: 1->0 W: 24
Node 2: 2->4 W: 52
Node 2: 2->1 W: 47
Node 2: 2->0 W: 95
Node 3: 3->4 W: 45
Node 3: 3->0 W: 3
Node 4: 4->3 W: 45
Node 4: 4->2 W: 52
Node 4: 4->0 W: 3

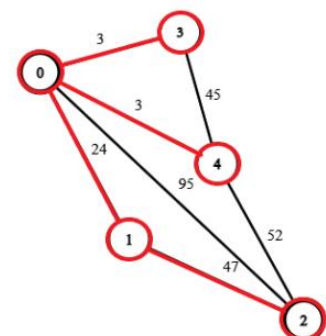
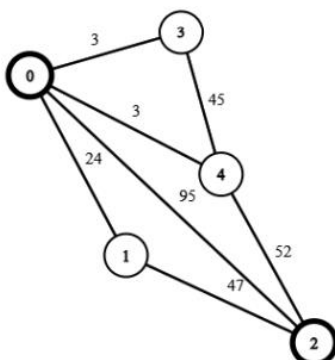
```

```

Prim :
The algorithm with the Adjacency Lists gives the following minimum spanning tree:
0 <--> 1 :weight: 24
1 <--> 2 :weight: 47
0 <--> 3 :weight: 3
0 <--> 4 :weight: 3
-----
Kruskal :

Edges minimum spanning tree created with Kruskal algorithm :
0 <--> 4      Weight: 3
0 <--> 3      Weight: 3
1 <--> 0      Weight: 24
1 <--> 2      Weight: 47
-----
Reverse-delete :
=====
Minimum Spanning Tree Edges:
1 <--> 2      Weight: 47
0 <--> 1      Weight: 24
0 <--> 4      Weight: 3
0 <--> 3      Weight: 3

```



Graph 3:

```

===== Graph#3 =====
Node 0: 0->3 W: 83
Node 0: 0->2 W: 49
Node 0: 0->1 W: 1
Node 1: 1->4 W: 84
Node 1: 1->3 W: 63
Node 1: 1->2 W: 80
Node 1: 1->0 W: 1
Node 2: 2->3 W: 92
Node 2: 2->1 W: 80
Node 2: 2->0 W: 49
Node 3: 3->4 W: 54
Node 3: 3->2 W: 92
Node 3: 3->1 W: 63
Node 3: 3->0 W: 83
Node 4: 4->3 W: 54
Node 4: 4->1 W: 84

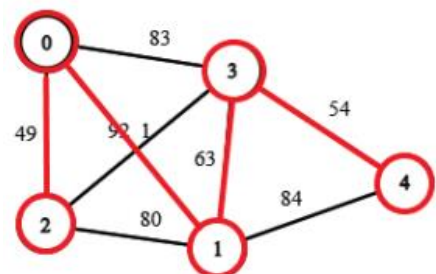
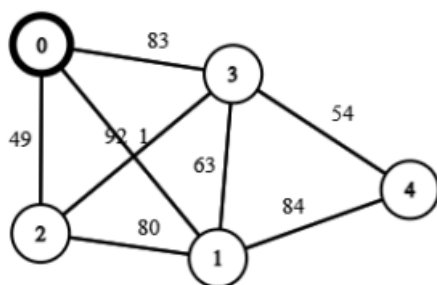
```

```

Prim :
The algorithm with the Adjacency Lists gives the following minimum spanning tree:
0 <--> 1 :weight: 1
0 <--> 2 :weight: 49
1 <--> 3 :weight: 63
3 <--> 4 :weight: 54
-----
Kruskal :

Edges minimum spanning tree created with Kruskal algorithm :
0 <--> 1      Weight: 1
2 <--> 0      Weight: 49
3 <--> 4      Weight: 54
3 <--> 1      Weight: 63
-----
Reverse-delete :
=====
Minimum Spanning Tree Edges:
1 <--> 3      Weight: 63
3 <--> 4      Weight: 54
0 <--> 2      Weight: 49
0 <--> 1      Weight: 1

```



Graph 4:

```

===== Graph#4 =====
Node 0: 0->4 W: 34
Node 0: 0->3 W: 37
Node 0: 0->2 W: 40
Node 0: 0->1 W: 30
Node 1: 1->4 W: 91
Node 1: 1->3 W: 25
Node 1: 1->2 W: 75
Node 1: 1->0 W: 30
Node 2: 2->4 W: 36
Node 2: 2->3 W: 13
Node 2: 2->1 W: 75
Node 2: 2->0 W: 40
Node 3: 3->4 W: 12
Node 3: 3->2 W: 13
Node 3: 3->1 W: 25
Node 3: 3->0 W: 37
Node 4: 4->3 W: 12
Node 4: 4->2 W: 36
Node 4: 4->1 W: 91
Node 4: 4->0 W: 34

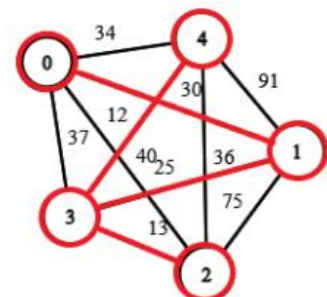
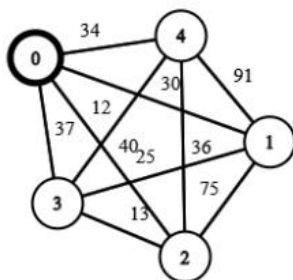
```

```

Prim :
The algorithm with the Adjacency Lists gives the following minimum spanning tree:
0 <--> 1 :weight: 30
3 <--> 2 :weight: 13
1 <--> 3 :weight: 25
3 <--> 4 :weight: 12
-----
Kruskal :

Edges minimum spanning tree created with Kruskal algorithm :
4 <--> 3      Weight: 12
3 <--> 2      Weight: 13
3 <--> 1      Weight: 25
1 <--> 0      Weight: 30
-----
Reverse-delete :
=====
Minimum Spanning Tree Edges:
0 <--> 1      Weight: 30
1 <--> 3      Weight: 25
2 <--> 3      Weight: 13
3 <--> 4      Weight: 12

```



Πλεονεκτήματα prim με λίστες γειτνίασης:

Πολυπλοκότητα χώρου:

Μια αναπαράσταση λίστας γειτνίασης απαιτεί συνήθως λιγότερη μνήμη σε σχέση με έναν πίνακα γειτνίασης, πιο συγκεκριμένα για αραιά γραφήματα. Ειδικότερα, σε μια λίστα γειτνίασης, απαιτείται μόνο να αποθηκεύσουμε πληροφορίες σχετικά με τις ακμές που υπάρχουν, σε αντίθεση με έναν πίνακα γειτνίασης όπου απαιτεί χώρο για κάθε πιθανή άκρη στο γράφημα. Το πλεονέκτημα εξοικονόμησης χώρου γίνεται πιο σημαντικό σε μεγαλύτερα γραφήματα ή πιο αραιά.

Ταχύτερη επανάληψη:

Όταν χρησιμοποιούμε μια λίστα γειτνίασης, η επανάληψη πάνω από τους γείτονες μιας κορυφής είναι συνήθως ταχύτερη σε σύγκριση με έναν πίνακα γειτνίασης. Σε μια λίστα γειτνίασης, οι γείτονες μιας κορυφής είναι άμεσα προσβάσιμοι, ενώ σε έναν πίνακα γειτνίασης, πρέπει να προσπελάσουμε μια ολόκληρη σειρά ή στήλη για να βρούμε τις γειτονικές κορυφές, γεγονός που απαιτεί περισσότερο χρόνο για μεγάλα γραφήματα.

Σημείωση: Η επεξήγηση των αλγορίθμων φαίνεται με αναλυτικά σχόλια μέσα στους κώδικες.

Γράφημα και αποτελέσματα για χρόνο εκτέλεσης αλγορίθμων:

Τρέχουμε το παράδειγμα μας για 4 γράφους, ο πρώτος θα έχει 150 nodes και θα αφαιρεθεί το 20% αυτών, ενώ από εκεί και έπειτα για τους άλλους θα έχουμε numVertices = numVertices.

```
-----
0 xronos poy etreje o Prim thn 1 fora einai :(prim)0.169000 ms
-----
0 xronos poy etreje o Kruskal thn 1 fora einai :(kruskal)0.162000 ms
-----
0 xronos poy etreje o Reverse-delete thn 1 fora einai :(Reverse-delete)30.938000 ms
-----
0 xronos poy etreje o Prim thn 2 fora einai :(prim)0.167000 ms
-----
0 xronos poy etreje o Kruskal thn 2 fora einai :(kruskal)0.160000 ms
-----
0 xronos poy etreje o Reverse-delete thn 2 fora einai :(Reverse-delete)29.832000 ms
-----
0 xronos poy etreje o Prim thn 3 fora einai :(prim)0.161000 ms
-----
0 xronos poy etreje o Kruskal thn 3 fora einai :(kruskal)0.155000 ms
-----
0 xronos poy etreje o Reverse-delete thn 3 fora einai :(Reverse-delete)30.641000 ms
-----
0 xronos poy etreje o Prim thn 4 fora einai :(prim)0.161000 ms
-----
0 xronos poy etreje o Kruskal thn 4 fora einai :(kruskal)0.156000 ms
-----
0 xronos poy etreje o Reverse-delete thn 4 fora einai :(Reverse-delete)30.772000 ms
-----
Average time of prim is:(prim)0.164500 ms
Average time of kruskal is:(kruskal)0.158250 ms
Average time of Reserve delete is:(Reverse-delete)30.545750 ms
```

